AI FOR RESEARCH MANAGEMENT · ELTE

The Course Glossary

A dictionary handbook of the key terms, for participants to keep and look things up in

125 terms in 7 sections · with an A–Z index

Edition: October 2026. Tool features and free-tier limits change often — check before relying on them.

How to use this handbook

This glossary collects the general terms used throughout the course — how language models work, how to prompt them, how to chain tools into workflows, the 7 Levels framework, the knowledge management system you build, the data-protection rules every step follows, and the core tools. Terms that belong only to a specific demo or use case are left out.

Terms are grouped by theme so that related ideas sit next to each other; within a section they follow the order in which the ideas build on one another. To look up a single word, use the **A–Z index** at the back.

Each entry gives a short definition. Where it helps, an *In practice* line says how the idea is used in daily work, and a → See also line points to related entries (clickable in the PDF).

Contents

1	How language models work (18 terms)	3
2	Prompting techniques (15 terms)	5
3	Workflows, tools and agents (16 terms)	7
4	The 7 Levels of AI (8 terms)	9
5	Knowledge management (28 terms)	10
6	Data protection and responsible use (24 terms)	12
7	Tools and features (16 terms)	14
	A–Z index	16

SECTION 1

How language models work

The mental model behind every tool in the course: what an LLM is, what it sees, and why it behaves the way it does.

Large language model (LLM)

A program that produces text by repeatedly predicting the most plausible next piece of text. Under the chat screen it is one very large file of numbers (parameters); it holds patterns learned from text, not stored pages or a rule book.

IN PRACTICE *Treat it as "powerful — but weird": excellent at messy, language-shaped work, unreliable as a source of facts.*

→ See also: Parameters, Hallucination, Token

Parameters

The billions of stored numbers (connection strengths, or weights) that make up a model. Training sets them; answering a question only reads them.

→ See also: Pre-training, Large language model (LLM)

Pre-training

The long, expensive first phase of building a model: it plays guess-the-next-word against an enormous body of text and every wrong guess nudges its parameters. The model keeps the patterns, not the pages — which is why it can sound confident yet misremember.

→ See also: Post-training, Training cutoff

Post-training

The shorter second phase that teaches a pre-trained model to behave like an assistant: answer questions, follow instructions, admit uncertainty, decline nonsense.

→ See also: Pre-training, System prompt

Token

The unit a model actually reads and writes. Text is chopped into numbered fragments: common words are usually one token, rare words several. Usage limits, prices and the context window are all counted in tokens, not words or characters.

IN PRACTICE *Tokenisation is why models stumble over letter counting and word games. A tokenizer playground lets you watch a sentence being split.*

→ See also: Context window, Reasoning tokens

Context window

The model's finite working memory: everything in the conversation — your prompts, pasted documents, uploaded files and the model's own replies — must fit inside a hard token limit. When a long chat overflows it, the oldest parts silently drop out.

IN PRACTICE *Start a fresh chat per task, and curate what you put in: more context is not better context.*

→ See also: Lost in the middle, Context engineering, Data minimisation

Lost in the middle

The tendency of models to weigh the beginning and end of a long input or conversation most heavily and to neglect what sits in the middle.

IN PRACTICE *A reason to keep inputs short and relevant, and to put the key instruction at the start or end.*

→ See also: Context window

Messages

A conversation is not one text but a stack of messages: an invisible system prompt at the top, then alternating user and assistant turns. The whole stack is re-read by the model at every turn.

→ See also: System prompt, Statelessness

System prompt

Standing instructions placed at the top of every conversation, usually invisible to the user, that shape everything the model does. When system and user instructions conflict, the system prompt usually wins.

IN PRACTICE *The "instructions" you write for a Gem or a custom agent are a system prompt.*

→ See also: Gem, Custom agent, Harness

Reasoning model

A model that "thinks out loud" in hidden reasoning tokens before it writes its answer. The pause before a good answer is this hidden work.

→ See also: Reasoning tokens

Reasoning tokens

The hidden tokens a reasoning model generates while working through a problem before answering. You wait for them and they count against limits, even though you never see them.

→ See also: Reasoning model, Token

Non-determinism (sampling)

The same question can produce different answers on different runs. The model's real output is a ranked list of candidate next tokens, and the next one is drawn with a little deliberate chance rather than always taking the top one.

IN PRACTICE *Variation between runs is normal, not an error. For work that must come out identical every time, use a template or checklist instead.*

→ See also: Deterministic-first principle

Statelessness

A model remembers nothing between conversations. Each chat starts blank unless the tool itself feeds earlier context back in (a Gem, saved info, a project, a memory feature).

→ See also: Persistent context, Gem

Training cutoff

The date after which the model has no knowledge, because its training text stops there. Anything newer needs a tool that searches the web.

→ See also: Perplexity, Deep Research, Hallucination

Hallucination

Fluent, confident output that is false — most often numbers, citations, names and dates. It follows from how models work: they produce plausible text, not verified fact.

IN PRACTICE *Verify anything you would sign your name to. Source-grounded tools reduce the risk by attaching citations you can check.*

→ See also: Source grounding, Critical checking, Sycophancy

Sycophancy

The model's tendency to agree with you, praise your draft or follow your framing instead of pushing back.

IN PRACTICE *Ask explicitly for weaknesses (critique-then-improve) or have the output attacked by a second prompt or model.*

→ See also: Critique-then-improve, Cross-model check

Embeddings

Numerical representations of text that place passages with similar meaning close together. They let software find related notes by meaning rather than by matching words (semantic search).

→ See also: Smart Connections, Copilot for Obsidian

Multimodal input

The ability of a tool to take more than text: PDFs and other files, images and screenshots, audio or video captions.

IN PRACTICE *File upload is still bound by the context window and by the anonymise-before-upload rule.*

→ See also: Context window, Anonymise-before-upload rule

SECTION 2

Prompting techniques

How to ask. The structures, shortcuts and patterns that take a user from Level 1 to Level 2 and beyond.

Prompt

The input you give a model in a single message: your instruction plus whatever context and constraints come with it. Output quality scales with input quality.

→ See also: ICC prompt structure, Prompt engineering

Prompt engineering

The practice of deliberately designing prompts — structure, context, examples, constraints — to get consistently good results, and iterating on them when they miss.

→ See also: ICC prompt structure, Prompt patterns, Meta-prompting

ICC prompt structure

The course's basic prompt shape: Instruction (what to do), Context (who you are, the situation, the audience) and Constraints (length, format, tone, what to avoid). Add an example of the output you want when you have one.

IN PRACTICE *Example skeleton: [Instruction] Explain… and list… [Context] I am… my audience is… [Constraints] Max. 600 words, use a table, plain language.*

→ See also: Prompt, Example-driven prompting (few-shot)

Ask-me-first

A shortcut: end the prompt with "Before answering, ask me the questions you need to give the best possible answer." The model interviews you and gathers the context it needs.

→ See also: Reverse prompt, Meta-prompting

Reverse prompt

A shortcut used after a long back-and-forth: "Write the single prompt that would have got us to this result on the first try." Save it for reuse.

IN PRACTICE *Reverse prompts make good standing instructions for a Gem.*

→ See also: Prompt library, Gem

Meta-prompting

Letting the model write the prompt: describe the task and ask for the optimal prompt instead of the answer, then fill in the specifics and run it. The model is often a better prompt engineer than you.

→ See also: Prompt Coach, Ask-me-first

Prompt patterns

Reusable prompt shapes that solve recurring problems. The course teaches four: persona, example-driven (few-shot), step-by-step and critique-then-improve. Every saved prompt is one of these shapes filled in.

→ See also: Persona pattern, Example-driven prompting (few-shot), Step-by-step prompting, Critique-then-improve

Persona pattern

Asking the model to act in a role ("Act as an experienced grants advisor reviewing…"). The role sets vocabulary, priorities and depth.

→ See also: Prompt patterns

Example-driven prompting (few-shot)

Giving the model one or more examples of the output you want and asking for the same style or structure. A good example beats a paragraph of description.

→ See also: Prompt patterns

Step-by-step prompting

Forcing intermediate steps before the conclusion ("first list the criteria, then check each, then conclude"). Improves reasoning on rule-checking tasks.

→ See also: Prompt patterns, Reasoning model

Critique-then-improve

Asking for a diagnosis before a repair: "First list the three biggest weaknesses, then rewrite fixing only those." Keeps the model from rewriting what already works and counters sycophancy.

→ See also: Prompt patterns, Sycophancy

Iteration

When an answer misses the mark, don't accept it: rewrite the prompt with more context or tighter constraints and try again. The habit that gets users to Level 2.

→ See also: Prompt engineering

Prompt library

Your saved collection of prompts that worked (reverse prompts, meta-prompts, improved prompts), kept for reuse. In the course it lives in your knowledge management system.

→ See also: Reverse prompt, KM System (knowledge management system)

Context engineering

Deciding what goes into the context window — which sources, instructions and memory — and what is left out. The second layer of the AI engineering stack, beyond the single prompt.

IN PRACTICE *Fewer, better tokens win: relevance beats volume.*

→ See also: Context window, AI engineering stack, Persistent context

Cross-model check

Running the same question, or an attack on an answer, through a second model and comparing. Disagreement flags what needs checking.

→ See also: Critical checking, Hallucination

SECTION 3

Workflows, tools and agents

From single prompts to chains of tools, and from chains to agents that act on their own.

AI engineering stack

A four-layer map of everything called "AI engineering", each layer wrapping the one below: prompt (the single message), context (what is in the window), harness (the standing setup around the model) and loop (the model running itself until a job is done).

→ See also: Prompt, Context engineering, Harness, Agentic loop

Harness

The standing setup around a model: its instructions (system prompt), the tools it may use and the format it must output. A Gem or a custom agent is a harness.

→ See also: AI engineering stack, Custom agent, System prompt

Agentic loop

A model running itself in cycles — plan, act, observe, decide the next step — until the job is done, instead of answering once and stopping.

→ See also: Agent, AI engineering stack

Tools (tool use)

Capabilities a model can call to act rather than just write: search the web, read a file, run code, send a message. A model with tools can do things in the world.

→ See also: Agent

Agent

An LLM with tools, running in a loop and choosing its own next step until the goal is reached. Level 5–6 territory in the 7 Levels framework.

→ See also: Tools (tool use), Agentic loop, Custom agent

Custom agent

An agent you configure yourself without code: you write its instructions (in effect a system prompt), give it knowledge and tools, and publish it for a recurring job.

→ See also: Agent Builder, Harness

Agent Builder

The no-code tool in Microsoft 365 Copilot for building custom agents. Writing an agent's instructions is prompting (ICC) applied to agent design.

→ See also: Custom agent, Agent Store

Agent Store

The catalogue of ready-made agents and apps available inside Microsoft 365 Copilot Chat, such as Prompt Coach and Study and Learn.

→ See also: Copilot Chat (Basic), Prompt Coach, Study and Learn

Workflow

A repeatable sequence of steps that turns an input into a finished output. In the course, a workflow is written down step by step so that each step can be routed, run and checked.

→ See also: Workflow orchestration, Blueprint

Workflow orchestration

Chaining several tools into one multi-step workflow, using the right tool for each step, where every step's output becomes a later step's input. The Level 4 skill.

IN PRACTICE *A chain that cannot be drawn has not been thought through.*

→ See also: Blueprint, Level 4 — Tool Orchestrator

Blueprint

The written plan of a workflow: for each step the tool, the input, the output, how it was run and the lane it runs in; for each skipped or changed step, one line saying why.

→ See also: Workflow orchestration, Two-lane rule

Check gate

The rule that is not a step but applies between all steps: check every output before it becomes the next step's input, and log it.

→ See also: Critical checking, AI-use log

Automation

A process triggered on a schedule or by an event that runs end to end without you starting or supervising each step. The Level 5 move.

→ See also: Level 5 — Builder

Deterministic-first principle

If a task can be done deterministically, it should be: use a template, spreadsheet or checklist for work that must come out the same every time, and AI for messy, judgement-shaped work.

→ See also: Non-determinism (sampling)

Vibe coding

Describing software in plain language, letting AI write the code, and judging only the result. It is changing who can build tools, but it is not engineering.

→ See also: Agentic engineering

Agentic engineering

The disciplined counterpart of vibe coding: deliberately engineering all four layers (prompt, context, harness, loop) and checking every lap of the loop.

→ See also: Vibe coding, AI engineering stack

SECTION 4

The 7 Levels of AI

The course's map of AI maturity. The course takes participants from Level 1 to Level 4–5.

7 Levels of AI

A framework describing seven stages of AI use, from asking one-off questions to running an operation with AI agents. Each level is defined by what you do differently, not by intelligence or technical skill.

→ See also: Level 1 — Question Asker, Level 7 — AI-Run Operation

Level 1 — Question Asker

AI used as a fancier search engine: one-off questions, quick emails, little thought about how the question is asked.

→ See also: Level 2 — Prompt Crafter

Level 2 — Prompt Crafter

You notice that how you ask changes what you get. Structured prompts (instruction–context–constraints), examples, ask-me-first and reverse prompts, iteration.

→ See also: ICC prompt structure, Ask-me-first, Reverse prompt

Level 3 — Power User

Persistent context: dedicated workspaces that already know your domain, so you stop rebuilding context in every chat. You use the tools' deeper features and feel the differences between models.

→ See also: Persistent context, Gem

Level 4 — Tool Orchestrator

You stop asking "what can this chatbot do?" and start asking "what is the best tool for this step?": specialised tools chained together, such as a notebook for documents and cited search for research.

→ See also: Workflow orchestration, NotebookLM (Gemini Notebook)

Level 5 — Builder

You build your own tools and automations without code — custom agents, internal tools, workflows triggered by a schedule or event that run without you.

→ See also: Custom agent, Automation

Level 6 — Systems Builder

You see systems waiting to be built everywhere: complex agentic workflows and agents that orchestrate other agents, running in the background. Outlook only in this course.

→ See also: Agent, Agentic loop

Level 7 — AI-Run Operation

Aspirational: a workforce of AI agents running an operation around the clock while people focus on what only they can do.

→ See also: 7 Levels of AI

SECTION 5

Knowledge management

The architecture each participant builds: sources are distilled in a notebook, checked by a person, and kept as linked notes in a personal vault.

KM System (knowledge management system)

The personal architecture built in the course: sources → NotebookLM (the distiller) → Personal Vault in Obsidian (the single source of truth), with two vault plugins for finding and asking. The arrow into the vault is a person.

→ See also: Distiller, Personal Vault, Human-mediated transfer (human gate)

Persistent context

Context that is set up once and travels with every new chat — standing instructions, reference files, saved facts — so you stop re-explaining yourself. The Level 3 move.

→ See also: Gem, Saved info, Level 3 — Power User

Source grounding

Answering only from the sources you supplied, not from the model's general training. A grounded tool says so when the sources don't contain the answer instead of inventing one.

→ See also: Citation, NotebookLM (Gemini Notebook), Hallucination

Citation

A pointer from an answer to the exact passage of the source it rests on. The receipt that comes with the answer, and the thing you open to verify it.

IN PRACTICE *An answer without a citation does not become a note.*

→ See also: Source grounding, Source line

Distiller

NotebookLM's role in the KM System: turning a pile of sources into source-grounded, cited answers that become vault notes once a person has checked them.

→ See also: NotebookLM (Gemini Notebook), KM System (knowledge management system)

Single source of truth

The one place whose content is treated as authoritative; everything else either feeds it or reads from it. In the course this is the Personal Vault.

→ See also: Personal Vault

Personal Vault

A participant's local Obsidian vault: a folder of Markdown notes that is the single source of truth of their KM System and their visual thinking space. Four folders, one per entity type; every note has a source line and a status.

→ See also: Obsidian, Entity types, Note template

Entity types

The four kinds of note in the vault, each with its own folder: concepts (definitions), processes (ordered steps with owners and deadlines), guides (how-tos for a first-timer) and projects (aim, partners, scheme, timeline, obligations).

→ See also: Personal Vault, Question shapes

Question shapes

Standard notebook questions, one per entity type, that turn a notebook's answer into a vault-ready note: define a term; list a process's steps; write a how-to; summarise a project — each with a citation per item.

→ See also: Entity types, Citation

Human-mediated transfer (human gate)

The rule that nothing moves from the notebook into the vault automatically. A person reads the answer, checks it against its citation and pastes it with a source line. No automated sync.

→ See also: Source line, Status (draft / checked), Critical checking

Source line

The field on every vault note recording where its content came from: notebook name, source document and section, if any.

→ See also: Note template, Citation

Status (draft / checked)

The field on every vault note showing whether its text has been compared with the citation: draft until checked, then checked.

→ See also: Human-mediated transfer (human gate), Note template

Note template

The standard skeleton pasted into every new vault note: frontmatter with type, source, captured date and status, then the title, the checked text and a Related section of links.

→ See also: Frontmatter, Source line, Status (draft / checked)

Markdown
A plain-text format in which simple marks create structure (# for headings, - for lists, ** for bold). Readable by people and by any text editor, with no lock-in.

→ See also: Obsidian, Frontmatter

Frontmatter
A block of fields between --- lines at the top of a Markdown note (e.g. type, source, status), used to describe and filter notes.

→ See also: Note template, Markdown

Wikilink
A link between notes written as [[Note title]]. Clicking a link to a note that doesn't exist yet creates it.

→ See also: Backlinks, Linking discipline

Backlinks
The list of every note that links to the one you are reading; shown automatically in Obsidian.

→ See also: Wikilink, Graph View

Tags
Labels such as #deadline that cut across folders and let you group or filter notes by theme.

→ See also: Graph View

Linking discipline
The habit that makes a vault knowledge rather than storage: every concept links to the process it appears in, every process to its concepts and guide, the project note to everything. A note with no links is a file, not knowledge.

→ See also: Wikilink, Graph View, Smart Connections

Graph View
Obsidian's map of the whole vault as nodes (notes) and edges (links), filterable by folder and tag; a local graph shows one note's neighbourhood. Isolated nodes reveal missing links.

→ See also: Linking discipline, Backlinks

Obsidian Canvas
A freeform whiteboard in Obsidian where notes, cards and images are arranged spatially. Not the same as Gemini Canvas.

→ See also: Gemini Canvas, Obsidian

Vault QA
Asking questions over your own vault and getting answers assembled from your notes, with the notes cited — the vault queried the way a notebook is queried over sources. Provided by Copilot for Obsidian.

→ See also: Copilot for Obsidian, Source grounding

Semantic search
Search by meaning rather than by exact words, powered by embeddings. Surfaces related notes even when they share no keywords.

→ See also: Embeddings, Smart Connections

Studio outputs
The formats NotebookLM can generate from one set of sources: cited chat, reports (briefing, study guide, FAQ, timeline), mind map, Audio and Video Overviews, flashcards and quizzes, and Deep Research.

→ See also: NotebookLM (Gemini Notebook), Audio Overview, Mind map

Audio Overview
A podcast-style spoken discussion of your sources generated by NotebookLM; in interactive mode you can interrupt the hosts with a question.

→ See also: Studio outputs

Mind map
A NotebookLM output that draws the main topics of the sources and how they branch — a useful first sketch of which notes will link to which.

→ See also: Studio outputs, Linking discipline

Four-floor map
A way to place Google's tools by job: grounding (NotebookLM), frontier model (Gemini), specialist (Gems) and tools (where AI enters documents and mail). The course puts Obsidian on the fourth floor as the place where insight becomes durable, linked work.

IN PRACTICE *The edge is never the tool; it is the system you build.*

→ See also: NotebookLM (Gemini Notebook), Gemini, Gem

Find → ground → keep
The routing line for new information: a searching tool finds the source, the notebook grounds it with citations, the vault keeps the checked result.

→ See also: Perplexity, NotebookLM (Gemini Notebook), Personal Vault

SECTION 6

Data protection and responsible use

Which tool may see what, and the principles behind the rule. Applies to every step of every workflow.

Two-lane rule

The course's data-governance rule. Before writing any prompt, route the task by asking one question: does it involve sensitive or IP-sensitive institutional data? Yes → Sensitive Lane; no → Daily Driver. The lane is chosen per task, not per person or per day.

→ See also: Sensitive Lane, Daily Driver, Local-only tools

Sensitive Lane

Microsoft 365 Copilot Chat (Basic) with the institutional account — the only tool authorised for sensitive or IP-sensitive institutional data: internal administration, unsubmitted grant proposals, personnel information, strategic financial planning. Never used for general tasks a free tool does better.

→ See also: Copilot Chat (Basic), Two-lane rule

Daily Driver

Google Gemini (free tier) and the other free cloud tools (NotebookLM, Perplexity, Copilot for Obsidian): the default for all non-sensitive work. Only anonymised, fictional or public content goes in.

→ See also: Gemini, Two-lane rule, What must never enter the Daily Driver

Local-only tools

Tools that call no cloud model at all and keep everything on your device (Obsidian itself, Smart Connections). Because the vault holds only anonymised or fictional content, the lane question does not arise for them.

→ See also: Obsidian, Smart Connections

Sensitive data

Information whose disclosure could harm people or the institution: personal data, unsubmitted proposals and their budgets and partner lists, internal financial and strategic documents, unpublished results, contract research, anything under a confidentiality agreement.

→ See also: What must never enter the Daily Driver, Personal data (PII)

What must never enter the Daily Driver

Personal data of any kind; unsubmitted proposals and their drafts, budgets and partner lists; internal financial and strategic documents; unpublished results and raw data; industrial or contract research results; documents under a confidentiality or data-processing agreement; anything not anonymised that should be.

IN PRACTICE *If it fails the anonymisation test even for the Sensitive Lane, it does not go into an AI tool at all.*

→ See also: Sensitive Lane, Anonymisation

Personal data (PII)

Personally identifiable information: anything that identifies a person directly or indirectly — names with roles, contact details, evaluations, CVs, personnel matters.

→ See also: Anonymisation, GDPR

Anonymisation

Removing or replacing names, identifiers, unpublished amounts, dates and partner identities so a document can no longer be linked to a person or an unsubmitted proposal. Done by hand, on a copy, before any file reaches an AI tool.

→ See also: Anonymise-before-upload rule, Personal data (PII)

Anonymise-before-upload rule

A document containing personal or sensitive information is anonymised before it is uploaded to any AI tool — in either lane. The enterprise boundary of the Sensitive Lane does not remove this rule.

→ See also: Anonymisation, Two-lane rule

Data minimisation

Collect and process only the information a clear, designated purpose actually needs. Points the same way as "more context is not better context".

→ See also: Context engineering, Privacy by design

Privacy by design

Building data protection into a system or workflow from the start rather than bolting it on afterwards.

→ See also: Data minimisation

Responsible AI — four pillars

Data governance (what may be collected, where it may travel, who is accountable); algorithmic fairness (filtering systemic bias); transparency and explainability (what data was used and how a decision was reached); human control (systems supplement human judgement, never take over responsibility).

→ See also: Human oversight, Accountability

Algorithmic bias

Systematic unfairness in a model's outputs inherited from its training data or prompts, which can replicate historical inequalities as if they were findings.

→ See also: Responsible AI — four pillars

Transparency

Being able to say what data was used and how an AI-assisted result was produced; in practice, declaring AI use and documenting it.

→ See also: AI-use log

Accountability

Responsibility for a system's behaviour, audits and performance assigned to a person before the tool is used.

→ See also: Human oversight

Human oversight

The model supplements human judgement rather than replacing human responsibility: the tool assists, the person signs.

→ See also: Critical checking, Accountability

Critical checking

Checking every AI output before it is used or passed to the next step: facts against sources, numbers by recalculating, citations by opening them. "Verify anything you would sign your name to."

→ See also: Check gate, Hallucination, Citation

AI-use log

A record of the tool, date and purpose of each AI-assisted step. It serves as the AI-use declaration on a deliverable and makes AI use transparent.

→ See also: Transparency, Check gate

GDPR

The EU General Data Protection Regulation, which governs how personal data may be processed. AI deployments in Europe must comply with it.

→ See also: EU AI Act, Personal data (PII)

EU AI Act

Regulation (EU) 2024/1689, the EU's risk-based law on artificial intelligence. Institutional AI rules supplement it.

→ See also: GDPR, Institutional AI directives

Institutional AI directives

ELTE's written rules on AI use: the Rector's directive 4/2025 (X.28.) and, at faculty level, the Faculty of Informatics Dean's directive 1/2026 (III.26.). Key points: human oversight and final responsibility, transparency and documentation of AI use, no sensitive data in external AI tools without explicit permission, and critical checking of every output.

IN PRACTICE *Check for a newer version before relying on the details.*

→ See also: Human oversight, AI-use log, EU AI Act

Enterprise data protection (tenant boundary)

Under an institutional Microsoft 365 licence, prompts and responses stay inside the institution's tenant, are not used to train models and are covered by the same terms as other institutional data. This is why Copilot Chat is the Sensitive Lane.

→ See also: Sensitive Lane, Free tier

Free tier

The no-cost plan of a commercial tool. Limits (messages, uploads, reports) and features change often, and the provider may use submitted content to improve its models — which is why free tools are Daily Driver only.

→ See also: Daily Driver, API key

API key

A personal secret code that lets a program (such as an Obsidian plugin) call a model provider's service on your account. Keep it private; a free key from Google AI Studio follows free-tier data terms.

→ See also: Google AI Studio, Copilot for Obsidian

SECTION 7

Tools and features

The course's core tools and the features of them that come up repeatedly. Features change often: check the current free tier before relying on a limit.

Gemini

Google's AI assistant, used on the free tier as the course's Daily Driver: drafting, prompting, file and image upload, long-document analysis, research.

→ See also: Daily Driver, Gem, Gemini Canvas

Gem

A custom Gemini workspace with standing instructions and reference files, so every new chat starts already knowing your domain. The Level 3 move. A Gem organises behaviour; a notebook organises evidence.

→ See also: Persistent context, System prompt, NotebookLM (Gemini Notebook)

Gemini Canvas

A drafting surface inside a Gemini chat for structured documents (and simple interactive tools): generate a draft, then edit or regenerate it section by section. Not the same as Obsidian Canvas.

→ See also: Obsidian Canvas, Gemini

Deep Research

An agentic research mode (in Gemini and inside NotebookLM) that searches the web over several minutes and returns a cited report. In a notebook, the report lands as a new source. Free-tier use is capped, so write the question before spending a run.

→ See also: Training cutoff, Citation, Find → ground → keep

Guided Learning

Gemini's tutoring mode: it teaches step by step and asks you questions instead of handing over a direct answer.

→ See also: Gemini

Saved info

Standing facts about you that every Gemini chat sees — a lightweight form of persistent context.

→ See also: Persistent context, Gem

NotebookLM (Gemini Notebook)

Google's source-grounded notebook, renamed Gemini Notebook in July 2026. It answers only from the sources you add (PDFs, Docs, web pages, YouTube captions, audio, etc.), with a citation on every answer, and generates Studio outputs. The distiller in the KM System; Daily Driver.

IN PRACTICE *Use it when you are drowning in material, when precision matters, or when the same knowledge must move across formats.*

→ See also: Source grounding, Distiller, Studio outputs

Copilot Chat (Basic)

Microsoft 365 Copilot Chat under the institutional licence, opened in Teams or microsoft365.com: the Sensitive Lane. It does not read the institution's SharePoint or OneDrive by itself — bring the relevant content into the chat — and it underperforms free tools on general tasks.

→ See also: Sensitive Lane, Agent Store, Enterprise data protection (tenant boundary)

Prompt Coach

A Microsoft agent in Copilot Chat that critiques a prompt and suggests an improved version — meta-prompting as a packaged agent. Run a prompt through it before you use it at any step.

→ See also: Meta-prompting, Agent Store

Study and Learn

A Microsoft agent that explains a concept and generates flashcards and quizzes from it; needs a Microsoft 365 Education account.

→ See also: Agent Store

Perplexity

A search engine that answers with inline citations to current web pages. A fast, cited first answer on current questions; the cited page can then become a notebook source. Daily Driver.

→ See also: Find → ground → keep, Training cutoff

Obsidian

A free (for personal use) note-taking application whose notes are plain Markdown files in a folder on your own disk — no account, no cloud, no lock-in. Hosts the Personal Vault.

→ See also: Personal Vault, Markdown, Local-only tools

Community plugins

Add-ons for Obsidian written by third parties. Enabling them requires switching off Restricted mode; install only what you need from the official list and read each plugin's data note first.

→ See also: Smart Connections, Copilot for Obsidian

Smart Connections

An Obsidian plugin that shows notes related to the one you are reading, using a local embedding model: no API key, works offline, nothing leaves the machine. Its job is finding — the link you forgot to make.

→ See also: Embeddings, Semantic search, Linking discipline

Copilot for Obsidian

An Obsidian community plugin (unrelated to Microsoft Copilot) for chatting with a model inside Obsidian and for Vault QA, on your own free Gemini API key. Its job is asking. Daily Driver, never the Sensitive Lane.

→ See also: Vault QA, API key, Daily Driver

Google AI Studio

Google’s developer site, where a free Gemini API key can be created with a personal Google account.

→ See also: API key, Copilot for Obsidian

A–Z index

#

7 Levels of AI 9

A

Accountability 13
Agent 7
Agent Builder 7
Agent Store 7
Agentic engineering 8
Agentic loop 7
AI engineering stack 7
AI-use log 13
Algorithmic bias 13
Anonymisation 12
Anonymise-before-upload rule 12
API key 13
Ask-me-first 5
Audio Overview 11
Automation 7

B

Backlinks 11
Blueprint 7

C

Check gate 7
Citation 10
Community plugins 14
Context engineering 6
Context window 3
Copilot Chat (Basic) 14
Copilot for Obsidian 15
Critical checking 13
Critique-then-improve 5
Cross-model check 6
Custom agent 7

D

Daily Driver 12
Data minimisation 12
Deep Research 14
Deterministic-first principle 7
Distiller 10

E

Embeddings 4
Enterprise data protection (tenant boundary) 13
Entity types 10
EU AI Act 13
Example-driven prompting (few-shot) 5

F

Find → ground → keep 11
Four-floor map 11
Free tier 13
Frontmatter 11

G

GDPR 13
Gem 14
Gemini 14
Gemini Canvas 14
Google AI Studio 15
Graph View 11
Guided Learning 14

H

Hallucination 4
Harness 7
Human oversight 13
Human-mediated transfer (human gate) 10

I

ICC prompt structure 5
Institutional AI directives 13
Iteration 5

K

KM System (knowledge management system) 10

L

Large language model (LLM) 3
Level 1 — Question Asker 9
Level 2 — Prompt Crafter 9
Level 3 — Power User 9
Level 4 — Tool Orchestrator 9
Level 5 — Builder 9
Level 6 — Systems Builder 9
Level 7 — AI-Run Operation 9
Linking discipline 11
Local-only tools 12
Lost in the middle 3

M

Markdown 11
Messages 3
Meta-prompting 5
Mind map 11
Multimodal input 4

N

Non-determinism (sampling) 4
Note template 10
NotebookLM (Gemini Notebook) 14

O

Obsidian 14
Obsidian Canvas 11

P

Parameters 3
Perplexity 14
Persistent context 10
Persona pattern 5
Personal data (PII) 12
Personal Vault 10
Post-training 3
Pre-training 3
Privacy by design 12
Prompt 5
Prompt Coach 14
Prompt engineering 5
Prompt library 5
Prompt patterns 5

Q

Question shapes 10

R

Reasoning model 3
Reasoning tokens 3
Responsible AI — four pillars 12
Reverse prompt 5

S

Saved info 14
Semantic search 11
Sensitive data 12
Sensitive Lane 12
Single source of truth 10
Smart Connections 15
Source grounding 10
Source line 10
Statelessness 4
Status (draft / checked) 10
Step-by-step prompting 5
Studio outputs 11
Study and Learn 14
Sycophancy 4
System prompt 3

T

Tags 11
Token 3
Tools (tool use) 7
Training cutoff 4
Transparency 13
Two-lane rule 12

V

Vault QA 11
Vibe coding 8

W

What must never enter the Daily Driver 12
Wikilink 11
Workflow 7
Workflow orchestration 7